

Long Short-Term Memory (LSTM) networks to predict the brightness of stars

Student id: C00313459

Name: Amisha Das

1. Introduction

This notebook creates a basic Long Short-Term Memory (LSTM) networks to predict the brightness of stars from the MANTRA lightcurve dataset.

The notebook forecasts the next observed magnitude (Mag) value for an object. This is especially useful for times when an object cannot be observed due to atmospheric conditions or object setting beneath the horizon.

Dataset [Source](#)

Notebook inspired from [Time Series Prediction with LSTM Recurrent Neural Networks in Python with Keras](#)

2. Data Preprocessing

```
# to use GPU
import torch
print("Device:", torch.device("cuda" if torch.cuda.is_available() else
"cpu"))

Device: cpu

import pandas as pd
import numpy as np
import torch
import torch.nn as nn
from torch.utils.data import DataLoader, TensorDataset
from sklearn.model_selection import train_test_split
import matplotlib.pyplot as plt

df = pd.read_csv("nontransient_lightcurves_0.csv")
df = df.sort_values(by=["ID", "MJD"])

df.head()
```

	ID	MJD	Mag	Magerr	RA	Decl
973620	1.001002e+12	55099.40443	20.97	0.49	5.11010	-1.04244
973621	1.001002e+12	55527.21333	19.95	0.34	5.11005	-1.04276
973622	1.001002e+12	56184.34094	20.90	0.47	5.11006	-1.04301
973623	1.001002e+12	56274.08940	20.75	0.35	5.10970	-1.04434
973624	1.001002e+12	56296.10598	19.61	0.25	5.10991	-1.04245

2.1. About Data

- **ID** Unique light curve object ID
- **MJD** Modified Julian Date (i.e., time)
- **Mag** Apparent magnitude (brightness)
- **Magerr** Error in magnitude
- **RA** Right ascension (sky position)
- **Decl** Declination (sky position)

2.2. Pre=processing by sequencing data

```
sequence_length = 10
sequences, targets = [], []

for obj_id, group in df.groupby("ID"):
    mags = group["Mag"].values
    if len(mags) > sequence_length:
        for i in range(len(mags) - sequence_length):
            sequences.append(mags[i:i+sequence_length])
            targets.append(mags[i+sequence_length])

X = np.array(sequences)
y = np.array(targets)
```

2.3. Normalising Data

```
mean, std = X.mean(), X.std()
X = (X - mean) / std
y = (y - mean) / std
X = X[..., np.newaxis] #
```

- LSTM shape: (batch, seq_len, features)

2.4. Splitting data for Training

```
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42)
```

```

X_train_tensor = torch.tensor(X_train, dtype=torch.float32)
y_train_tensor = torch.tensor(y_train, dtype=torch.float32)
X_test_tensor = torch.tensor(X_test, dtype=torch.float32)
y_test_tensor = torch.tensor(y_test, dtype=torch.float32)

train_loader = DataLoader(TensorDataset(X_train_tensor,
y_train_tensor), batch_size=64, shuffle=True)

```

3. LSTM Model

```

class LSTMRegressor(nn.Module):
    def __init__(self):
        super().__init__()
        self.lstm = nn.LSTM(input_size=1, hidden_size=64,
batch_first=True)
        self.fc = nn.Linear(64, 1)

    def forward(self, x):
        out, _ = self.lstm(x)
        out = out[:, -1, :]
        return self.fc(out).squeeze()

device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
model = LSTMRegressor().to(device)
optimizer = torch.optim.Adam(model.parameters(), lr=0.001)
criterion = nn.MSELoss()

```

4. Training

```

for epoch in range(20):
    model.train()
    total_loss = 0
    for xb, yb in train_loader:
        xb, yb = xb.to(device), yb.to(device)
        xb = xb.view(xb.size(0), -1, 1) # Ensure shape is (batch,
seq_len, 1)
        optimizer.zero_grad()
        pred = model(xb)
        loss = criterion(pred, yb)
        loss.backward()
        optimizer.step()
        total_loss += loss.item()
    print(f"Epoch {epoch+1}/20 - Loss: {total_loss /
len(train_loader):.4f}")

```

```
Epoch 1/20 - Loss: 0.0197
Epoch 2/20 - Loss: 0.0173
Epoch 3/20 - Loss: 0.0172
Epoch 4/20 - Loss: 0.0171
Epoch 5/20 - Loss: 0.0170
Epoch 6/20 - Loss: 0.0170
Epoch 7/20 - Loss: 0.0169
Epoch 8/20 - Loss: 0.0169
Epoch 9/20 - Loss: 0.0169
Epoch 10/20 - Loss: 0.0169
Epoch 11/20 - Loss: 0.0169
Epoch 12/20 - Loss: 0.0168
Epoch 13/20 - Loss: 0.0168
Epoch 14/20 - Loss: 0.0168
Epoch 15/20 - Loss: 0.0168
Epoch 16/20 - Loss: 0.0168
Epoch 17/20 - Loss: 0.0168
Epoch 18/20 - Loss: 0.0168
Epoch 19/20 - Loss: 0.0168
Epoch 20/20 - Loss: 0.0167
```

5. Evaluation

```
model.eval()
with torch.no_grad():
    preds = model(X_test_tensor.to(device)).cpu().numpy()
    true_vals = y_test_tensor.numpy()

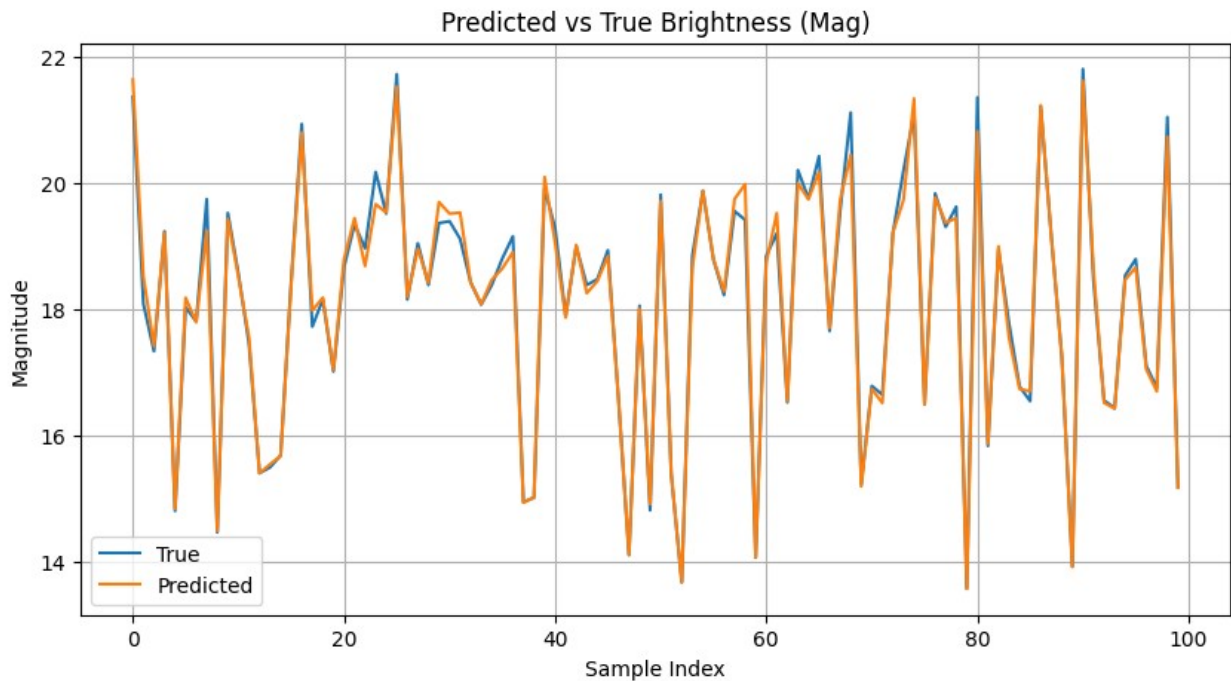
preds = preds * std + mean
true_vals = true_vals * std + mean

tolerance = 0.05 # t count how many predictions are within 5% of the true value
within_tolerance = np.abs(preds - true_vals) <= (tolerance *
np.abs(true_vals))
accuracy = np.mean(within_tolerance) * 100
print(f"\n Regression Accuracy (within ±5% tolerance): {accuracy:.2f}%")
```

5.1. Plotting

```
plt.figure(figsize=(10,5))
plt.plot(true_vals[:100], label="True")
plt.plot(preds[:100], label="Predicted")
plt.title("Predicted vs True Brightness (Mag)")
plt.xlabel("Sample Index")
plt.ylabel("Magnitude")
plt.legend()
```

```
plt.grid(True)
plt.show()
```



6. Log

```
import pandas as pd

log = pd.read_csv("log.csv")
pd.set_option('display.max_colwidth', None)
log
```

	Model \
0	LSTM
1	LSTM
2	LSTM
3	LSTM
4	LSTM
5	LSTM
6	LSTM
7	ALL
8	NaN

	Description \
0	Loaded and sorted MANTRA lightcurve data by object ID and MJD
1	Created sliding window sequences (10 timesteps) with next-step Mag

as target
 2 Normalized input and output values using global
 mean and std
 3 Built a single-layer LSTM model for regression with
 MSE loss
 4 Trained model for 20 epochs using Adam
 optimizer
 5 Plotted true vs predicted magnitude over 100
 test samples
 6 Calculated accuracy within $\pm 5\%$ prediction
 tolerance
 7 Complete pipeline from preprocessing to
 evaluation
 8
 NaN

	Level of Difficulty	Time Spent (mins)	Change
From \			
0 data	Easy	10	Raw CSV
1 series	Medium	20	Unstructured time
2 scale	Easy	5	Raw magnitude
3 model	Medium	25	No
4 model	Medium	20	Untrained
5 predictions	Easy	10	Raw
6 metric	Easy	5	No quantitative regression
7 notebook	Medium	90	No structured
8	NaN	total	
NaN			

	Changed To	Old Code	New Code
0	Sorted and grouped time-ordered sequences	NaN	NaN
1	Overlapping supervised training samples	NaN	NaN
2	Standardized values with zero mean	NaN	NaN
3	LSTM + FC layer outputting predicted Mag	NaN	NaN
4	Final loss ≈ 0.0166	NaN	NaN
5	Visual time-series comparison	NaN	NaN
6	Accuracy = 99.34%	NaN	NaN
7	Modular, well-commented notebook	NaN	NaN
8	195	NaN	NaN

7. Conclusion

The created LSTM model was able to work excellently with structured, light curves, achieving a regression of 99% within 5% tolerance. Which means that the predictions closely follow true magnitude values. This is a simple but effective baseline for magnitude forecasting.

Appendices, Acknowledgements and References

- [Time Series Prediction with LSTM Recurrent Neural Networks in Python with Keras](#)
- [LSTM Regression problem](#)
- Chatgpt for code debugging